Bookly support agent: think fast, act right

Demo Slides

Jonathan Klick · October 2026

✦ The thesis

Don't fake a human. Do what only an agent can do.

What I passed on

Simulate a human

Typing indicator

Built-in response delays

Lowercase and typos to seem real

Feels familiar, but it's slower and gives a false sense of a person.

What I chose

Lean into the agent

Instant: replies in seconds, not minutes; no queue

Mobile usage: single-handed, on the go

No guessing: refunds use fixed rules, not AI

Not built yet: suggestions ("Readers who liked it also loved [title]") and voice for returns or orders while driving or walking.

✦ Architecture

How an inquiry flows

1 · Input

Message

Typed text, or a tap on an order or kind of help

→

2 · Jev

Triage

What they want, is it clear, are they upset. Haiku steps in if Jev isn't responding

→

3 · Fixed rules

Code decides

Hand to a person, say it's out of scope, ask one question, or answer

→

4 · Claude Haiku

Reply

Writes the answer from tool results, with only that request's tools

---

Tools

Look up an order, quote a return, record a refund or exchange

Memory

The conversation and the signed-in shopper's orders, for the session

Prompts

Triage asks yes/no questions; one reply prompt that sticks to the facts

✦ Key decisions

Three notable choices

Chose	Traded off	Why it was worth it
Claude only steps in on clear requests	It can't improvise through a vague or off-topic message. Those get a set question, reply or handoff	Each reply gets only the tools it needs (a status check can't issue a refund). Zero wrong actions across 31 test cases
Fixed rules set every refund amount	No judgment calls. Anything the rules don't cover goes to a person	The agent can't make up a refund amount or promise one it shouldn't
A fast decision model (Jev) for triaging customer requests	A second, early-access vendor	Same accuracy as Claude, almost 6x faster (176 ms vs 1,009 ms). Confidence scores let me set a cutoff for each action based on risk

Source: Bookly test set, 31 scripted conversations · docs/benchmark.md · ADR 0002

✦ What I'd do differently

From demo to production

01 · State

Conversations live in one process

With several servers, the next message can land on one that's never seen the chat, and it quietly starts over. Enhancement: store it in Redis.

02 · Concurrency

Each turn holds a thread for seconds

Fine at demo scale. At real volume, the turn goes async or onto a job queue.

03 · Handoff

A person never actually sees it

Today it's a record in memory. Production opens a Zendesk ticket with the triage and tool results attached.

04 · Observability

Nobody can audit what the agent did

Today there's only the live panel. People need a searchable record of every chat, decision and refund to review, audit and report on.

✦ What I'd do differently

From demo to production

Layer	Today	Production
Entry point	Laptop, port 8000 and up	DNS → CDN and web firewall → load balancer (HTTPS, routing, rate limits)
App servers	One uvicorn process	Several containers (Kubernetes, ECS or Cloud Run), scaled on load
Conversation state	Python dict, lost on restart	Redis or a database, so any copy can serve any turn
Orders and returns	JSON file, returns in memory	Calls to the real order and returns systems
Model calls	Straight from the handler	Plus rate limits, timeouts, retries, spend caps; optional LLM gateway
Model backup	Anthropic only (Haiku)	A fast, cheap OpenAI GPT model takes over if Anthropic is down
Shopper identity	Picked from a menu	Real login; every tool call scoped to the signed-in shopper's orders
Secrets	Shell environment variables	A secrets manager
Visibility	The live panel	Logs, traces, per-turn cost and speed, transcripts for evals